6-Week C++ Systems-Oriented Track

A rigorous, practical syllabus for Computer Architecture, Operating Systems, Systems Programming, Distributed Systems, and CUDA.

Core Objective: By the end of this track, you will be able to design and implement a non-trivial C++ program from an empty directory, reason deeply about memory and ownership, write concurrent code, build/debug/test with CMake, and structure maintainable software.

Week 1: C++ Fundamentals

Foundation

Compilation vs linking; primitive types; control flow; functions; references; pointers; arrays; strings; structs/classes; const; stack vs heap; basic I/O; namespaces; header/source separation.

Project: CLI File/Text Utility — Read/process files, compute stats, write output across multiple header/source files.

Week 2: Memory, Ownership & Object Lifetime

Memory Safety

Constructors/destructors; RAII; ownership; dynamic allocation; unique_ptr/shared_ptr/weak_ptr; copy/move constructors & assignment; lvalues/rvalues; std::move; rule of 0/3/5; operator overloading.

Project: RAII-managed Buffer/Resource Library — Implement allocation, resizing, copying, moving, and automatic cleanup.

Week 3: STL & Generic Programming

Data Structures

vector, array, deque, list, map, unordered_map, set, queue, stack; iterators; algorithms; lambdas; function objects; optional; variant; tuple; string_view; basic templates; container complexity.

Project: In-memory Key-Value Store — SET/GET/DELETE/KEYS/SIZE commands, persistence, parsing, robust error handling, and unit tests.

Week 4: Concurrency

Parallelism

std::thread; mutexes; lock_guard; unique_lock; condition variables; futures/promises; atomics; race conditions; deadlocks; critical sections; producer-consumer; thread-safe APIs.

Project: Thread Pool — Worker threads, task queue, synchronization, and clean shutdown.

Week 5: Building Real C++ Software & Architecture

Tooling & Design

CMake targets/libraries/dependencies; Debug/Release builds; LLDB/GDB, stack traces, memory inspection; unit testing, benchmarking, profiling. Software design: separation of concerns, RAII, composition over inheritance, interface design.

Week 6: Capstone: Concurrent Data Processing Engine

Integration

Combine C++, STL, RAII, CMake, concurrency, synchronization, logging, tests, benchmarks, and documentation. Pipeline: input reader → thread pool workers → aggregation → output. Benchmark 1/2/4/8/16 threads.

Recommended Project Structure

```
my_project/ ├── CMakeLists.txt ├── README.md ├── include/my_project/ | └── processor.hpp ├── src/
```

Stage	Project	Primary Learning Focus
1	CLI File/Text Utility	Language fundamentals + compilation model
2	RAII Buffer Library	Memory management + object lifetime
3	Key-Value Store	STL utilization + API design
4	Thread Pool	Concurrency & synchronization
5	Concurrent Data Processing Engine	End-to-end integration & performance profiling